ความสัมพันธ์ของฐานข้อมูล (DB Relations)

ระบบตามที่สร้างจริงบนสาขา `mini-erp` (เวอร์ชัน 2.0): เริ่มจากตารางพื้นฐาน แล้วตามด้วยสิ่งที่ Purchase, QT 2.0 และ User 2.0 เพิ่มหรือเปลี่ยนแปลง ที่มา (Purchase Logic Draft2, QT Upgrade 2.0, User Version 2.0)

สรุปเวอร์ชันปัจจุบัน (ตามที่สร้างจริง)

23 ตารางใน 5 กลุ่ม ส่วนที่ 1 และ 2 ด้านล่างระบุคีย์และกฎทุกข้อ ส่วนนี้เป็นฉบับย่อ

กลุ่ม	ตาราง	ความสัมพันธ์หลัก
ผู้ใช้และไฟล์	User, Attachment	Attachment.uploaded_by → User; Attachment ชี้ไปยังเจ้าของ (PurchaseOrder, QtReference หรือ ProductionOrder) ด้วย `owner_type` + `owner_id` โดยไม่มีคีย์ในฐานข้อมูล
ข้อมูลหลัก	Customer, Product, Material, MaterialLot, ProductBom, Supplier, SupplierMaterial	Product → ProductBom → Material (หนึ่งรายการ BOM); Material → MaterialLot (สต็อกแยกตามล็อต); Supplier → SupplierMaterial → Material (สิ่งที่ผู้จัดจำหน่ายขาย พร้อมราคา)
งานขาย	QtReference, QtLine, ProductionOrder, OrderLine, OrderProcess, ProgressEntry	Customer → QtReference → QtLine → Product; การยืนยัน QT สร้าง ProductionOrder หนึ่งใบ (QtReference.production_order_id, หนึ่งต่อหนึ่ง); Order → OrderLine → OrderProcess → ProgressEntry (โดย User)
บัญชี	Invoice, InvoiceLine, Payment, Receipt	หนึ่ง Invoice ต่อหนึ่ง ProductionOrder; หนึ่ง Payment และหนึ่ง Receipt ต่อหนึ่ง Invoice
จัดซื้อและสต็อก	PurchaseOrder, PurchaseOrderLine, PurchasePayment, StockChange	Supplier → PurchaseOrder → PurchaseOrderLine → Material; หนึ่ง PurchasePayment ต่อหนึ่ง PO; StockChange (วัตถุดิบ ผู้ใช้ และใบสั่งผลิตหรือ PO ถ้ามี) คือประวัติการเปลี่ยนแปลงล็อตทุกครั้ง

กฎที่ควรจำ:

- **การลบ:** ข้อมูลลูกถูกลบไปพร้อมกับข้อมูลแม่ (รายการ ขั้นตอนการผลิต BOM ล็อต สินค้าของผู้จัดจำหน่าย ไฟล์แนบ) ส่วนอื่นที่ถูกอ้างอิงอยู่จะตอบ `409 IN_USE` ลบได้เฉพาะ QT ที่เป็น `Draft` หรือ PO ที่เป็น `Draft` เท่านั้น ใบแจ้งหนี้ ใบเสร็จรับเงิน และประวัติสต็อกจะไม่ถูกลบ
- **หนึ่งต่อหนึ่ง:** Invoice ต่อใบสั่งผลิต, Payment และ Receipt ต่อ Invoice, PurchasePayment ต่อ PO, ProductionOrder ต่อ QT ที่ยืนยันแล้ว
- **ไม่ซ้ำกัน:** User.email และรหัส/เลขที่ของ Product, Material, Supplier, QT, ใบสั่งผลิต, ใบแจ้งหนี้, ใบเสร็จรับเงิน และ PO; (วัตถุดิบ, เลขที่ล็อต); (ผู้จัดจำหน่าย, วัตถุดิบ); (QT, สินค้า)
- **สถานะ:** QT `Draft` → `Confirmed`; ใบสั่งผลิต `Draft` → `In Progress` → `Completed` (คำนวณ); ใบแจ้งหนี้ `Unpaid` → `Paid`; PO `Draft` → `Sent` → `Received`
- **สต็อกเคลื่อนไหว** ผ่าน StockChange + MaterialLot เท่านั้น: การเริ่มผลิตตัดตาม BOM (ล็อตเก่าก่อน) การรับ PO เพิ่มครั้งเดียว และผู้ใช้สามารถเพิ่ม ตัด หรือปรับล็อตด้วยมือได้

ส่วนที่ 1: ตารางพื้นฐาน (Refrigerator Factory ก่อนเวอร์ชัน 2.0)

รายการคีย์ ความสัมพันธ์ และกฎการลบ แยกตามตาราง แบบจำลองทั้งหมดตามที่สร้างจริงอยู่ในเอกสาร แผนภาพ ER (ER Diagram) แถวของ QtReference, Material และ User ถูกเปลี่ยนแปลงโดยส่วนที่ 2 ด้านล่าง ที่มา: docs/api (`master-data.md`, `stock.md`, `qt.md`, `orders.md`, `invoices.md`, `receipts.md`)

คีย์นอก (Foreign keys)

ตาราง	คอลัมน์	อ้างอิงไปยัง	ความสัมพันธ์เชิงจำนวน	กฎการลบ
QtReference	customer_id	Customer.id	หลายต่อหนึ่ง	ลูกค้าที่ถูกใช้อยู่ → `409 IN_USE`
ProductionOrder	qt_id	QtReference.id	หลายต่อหนึ่ง	QT ที่ถูกใช้โดยใบสั่งผลิต → `409 IN_USE`
ProductionOrder	customer_id	Customer.id	หลายต่อหนึ่ง	ลูกค้าที่ถูกใช้อยู่ → `409 IN_USE`
OrderLine	order_id	ProductionOrder.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับใบสั่งผลิต
OrderLine	product_id	Product.id	หลายต่อหนึ่ง	สินค้าที่ถูกใช้อยู่ → `409 IN_USE`
OrderProcess	line_id	OrderLine.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับรายการ
ProgressEntry	process_id	OrderProcess.id	หลายต่อหนึ่ง	ไม่ถูกลบ
ProgressEntry	user_id	User.id	หลายต่อหนึ่ง	ผู้ใช้ที่มีประวัติ → `409 IN_USE`
ProductBom	product_id	Product.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับสินค้า
ProductBom	material_id	Material.id	หลายต่อหนึ่ง	วัตถุดิบที่อยู่ใน BOM → `409 IN_USE`
MaterialLot	material_id	Material.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับวัตถุดิบ
StockChange	material_id	Material.id	หลายต่อหนึ่ง	วัตถุดิบที่มีการเปลี่ยนแปลงนอกเหนือจากสต็อกแรกจะลบไม่ได้
StockChange	order_id	ProductionOrder.id	หลายต่อหนึ่ง (ไม่บังคับ)	กำหนดเมื่อการเริ่มผลิตตัดสต็อก
StockChange	user_id	User.id	หลายต่อหนึ่ง	ไม่ถูกลบ
ใบแจ้งหนี้	order_id	ProductionOrder.id	หนึ่งต่อหนึ่ง	หนึ่งใบแจ้งหนี้ต่อหนึ่งใบสั่งผลิต
InvoiceLine	invoice_id	Invoice.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับใบแจ้งหนี้
การจ่ายเงิน	invoice_id	Invoice.id	หนึ่งต่อหนึ่ง	รับชำระเต็มจำนวนหนึ่งครั้งต่อหนึ่งใบแจ้งหนี้
Receipt	invoice_id	Invoice.id	หนึ่งต่อหนึ่ง	หนึ่งใบเสร็จรับเงินต่อหนึ่งใบแจ้งหนี้

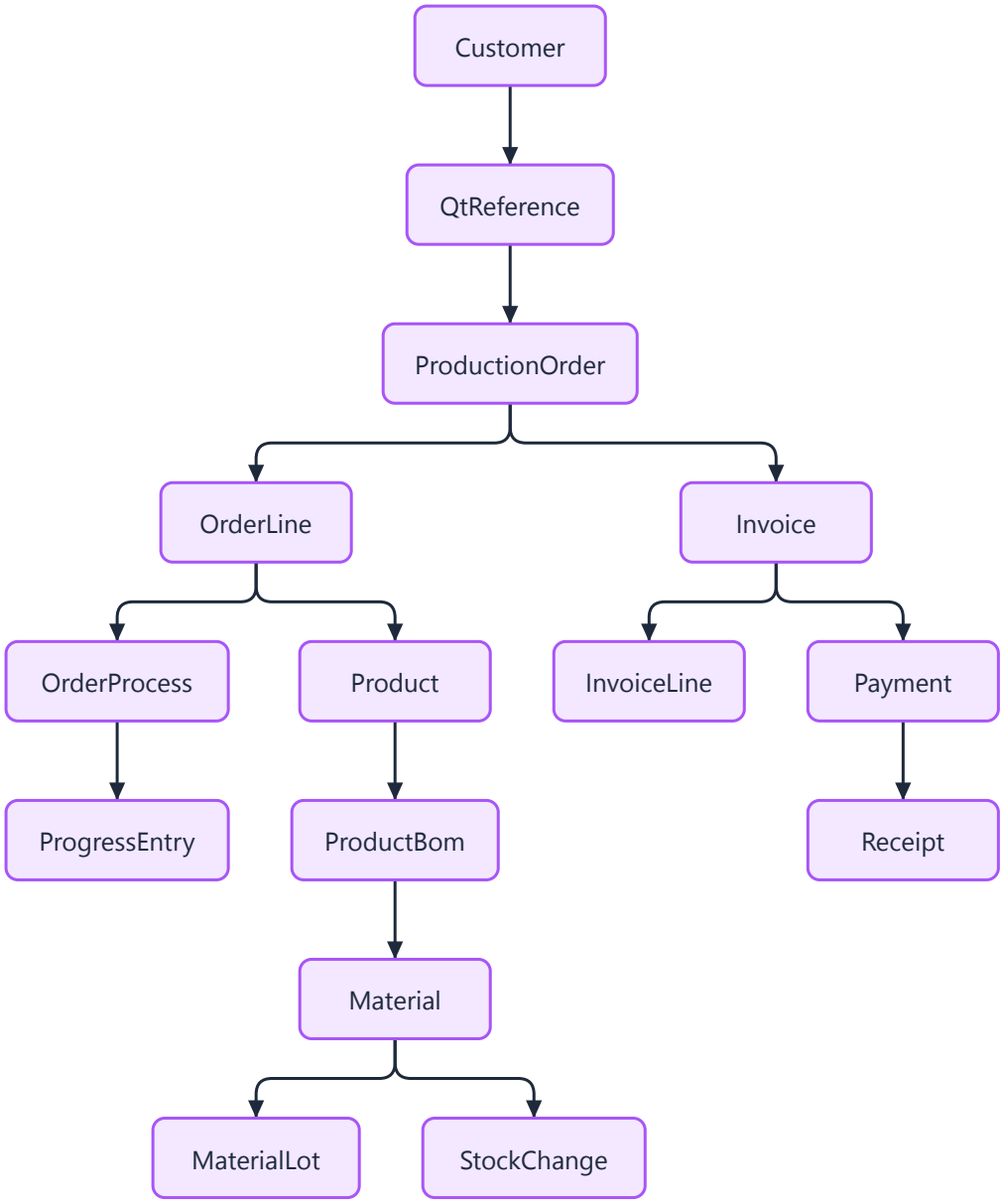
คีย์ที่ไม่ซ้ำกัน (Unique)

ตาราง	ค่าที่ไม่ซ้ำ
User	email (ตัวพิมพ์เล็ก)
Product	code
วัตถุดิบ	code
MaterialLot	(material_id, lot_number)
QtReference	qt_number
ProductionOrder	order_number (`ORD-yyyy-0001` เซิร์ฟเวอร์เป็นผู้กำหนด)
ใบแจ้งหนี้	invoice_number, order_id
การจ่ายเงิน	invoice_id
Receipt	receipt_number, invoice_id

กฎของวงจรชีวิตที่เกี่ยวกับหลายตาราง

เหตุการณ์	ตารางที่ถูกเขียน	กฎ
เริ่มผลิต (`POST /orders/:id/start`)	ProductionOrder.started_at, MaterialLot, StockChange	ตัดวัตถุดิบตาม BOM ของทุกรายการ โดยตัดล็อตเก่าก่อน ถ้าวัตถุดิบไม่พอจะตอบ `409 STOCK_SHORT` และล็อกสินค้าและขั้นตอนการผลิต
บันทึกความคืบหน้า	ProgressEntry, OrderProcess	อัปเดตสถานะและจำนวนที่ทำเสร็จของขั้นตอนการผลิต ใบสั่งผลิตจะเสร็จสิ้นเมื่อทุกขั้นตอนเสร็จสิ้น
ออกใบแจ้งหนี้	Invoice, InvoiceLine	ทำได้เฉพาะใบสั่งผลิตที่เสร็จสิ้น และทำได้ครั้งเดียว คัดลอกข้อมูลไว้ (snapshot) และล็อกความคืบหน้าของใบสั่งผลิต
รับชำระใบแจ้งหนี้	Payment, Invoice.status, Receipt	วันที่และจำนวนเงินที่รับชำระต้องเท่ากับยอดรวม (`MUST_EQUAL_TOTAL`) ตั้งสถานะ `Paid` และออกใบเสร็จรับเงินหนึ่งใบ
ปรับสต็อกด้วยมือ	MaterialLot, StockChange	`Add`, `Deduct` หรือ `Adjust` ล็อตได้ทุกเมื่อ ประวัติจะไม่ถูกแก้ไขหรือลบ

ห่วงโซ่ของการขายหนึ่งครั้ง



รูปที่ 1. ห่วงโซ่ของการขายหนึ่งครั้ง

ส่วนที่ 2: สิ่งที่เพิ่มหรือเปลี่ยนในเวอร์ชัน 2.0 (ตามที่สร้างจริง)

คีย์ ความสัมพันธ์ และกฎ หลังการเปลี่ยนแปลงของ Purchase, QT 2.0 และ User 2.0 แสดงเฉพาะแถวที่ **เพิ่มหรือเปลี่ยน** ส่วนอื่นเหมือนในส่วนที่ 1 สถานะของใบสั่งผลิตคือ `Draft`, `In Progress`, `Completed` (คำนวณ; `Draft` ใช้แทน `New`) ใบสั่งผลิตเกิดจากการยืนยัน QT เท่านั้น

คีย์นอกใหม่

ตาราง	คอลัมน์	อ้างอิงไปยัง	ความสัมพันธ์เชิงจำนวน	กฎการลบ
SupplierMaterial	supplier_id	Supplier.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับผู้จัดจำหน่าย

ตาราง	คอลัมน์	อ้างอิงไปยัง	ความสัมพันธ์เชิงจำนวน	กฎการลบ
SupplierMaterial	material_id	Material.id	หลายต่อหนึ่ง	วัตถุดิบที่ผู้จัดจำหน่ายขายอยู่ → 409 `IN_USE`
PurchaseOrder	supplier_id	Supplier.id	หลายต่อหนึ่ง	ผู้จัดจำหน่ายที่มี PO → `409 IN_USE`
PurchaseOrder	created_by	User.id	หลายต่อหนึ่ง	ผู้ใช้ที่มีประวัติ → 409 `IN_USE`
PurchaseOrderLine	po_id	PurchaseOrder.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับ PO; ลบได้เฉพาะ PO ที่เป็น `Draft`
PurchaseOrderLine	material_id	Material.id	หลายต่อหนึ่ง	วัตถุดิบที่อยู่ใน PO → `409 IN_USE`
PurchasePayment	po_id	PurchaseOrder.id	หนึ่งต่อหนึ่ง	การจ่ายเงินหนึ่งครั้งต่อหนึ่ง PO
StockChange	purchase_order_id	PurchaseOrder.id	หลายต่อหนึ่ง (ไม่บังคับ)	กำหนดเมื่อรับของตาม PO
QtLine	qt_id	QtReference.id	หลายต่อหนึ่ง	ถูกลบไปพร้อมกับ QT; ลบได้เฉพาะ QT ที่เป็น `Draft`
QtLine	product_id	Product.id	หลายต่อหนึ่ง	สินค้าที่อยู่ใน QT → `409 IN_USE`
QtReference	production_order_id	ProductionOrder.id	หนึ่งต่อหนึ่ง (ไม่บังคับ)	กำหนดเมื่อยืนยัน QT
Attachment	uploaded_by	User.id	หลายต่อหนึ่ง	ไม่ถูกลบ
Attachment	(owner_type, owner_id)	PurchaseOrder, QtReference หรือ ProductionOrder	หลายต่อหนึ่ง **ไม่มีคีย์ในฐานข้อมูล**	ถูกลบไปพร้อมกับเจ้าของ (service ลบไฟล์ด้วย)

คอลัมน์ที่เปลี่ยน

ตาราง	การเปลี่ยนแปลง
QtReference	เพิ่ม `status` (`Draft`, `Confirmed`), `customer_po_number`, `customer_po_date`, `production_order_id`; ตัด `file_name`, `file_type`, `file_path` ออก (ย้ายไป Attachment)
วัตถุดิบ	เพิ่ม `unit`
User	กฎ: `role = Worker` ต้องมี `department` เป็น (`Sales`, `Factory`)

คีย์ที่ไม่ซ้ำกัน (Unique)

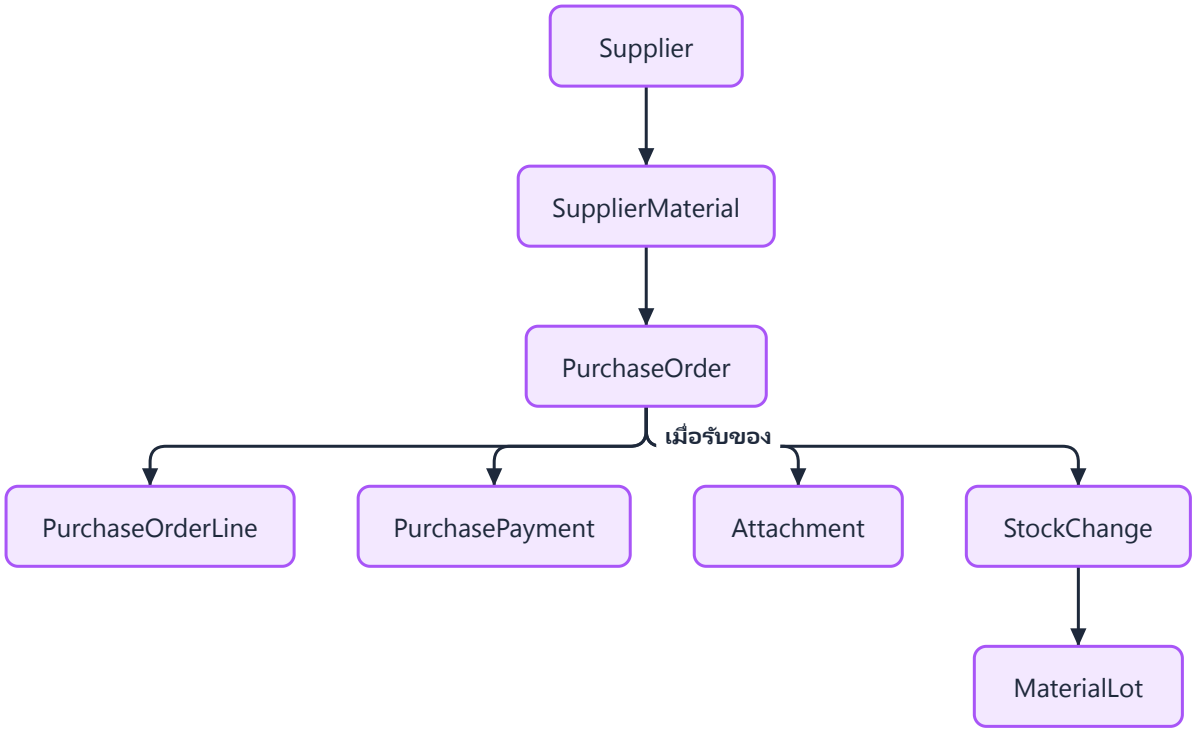
ตาราง	ค่าที่ไม่ซ้ำ
ผู้จัดจำหน่าย	code
SupplierMaterial	(supplier_id, material_id)
PurchaseOrder	po_number
PurchasePayment	po_id

ตาราง	ค่าที่ไม่ซ้ำ
QtReference	qt_number, production_order_id
QtLine	(qt_id, product_id)

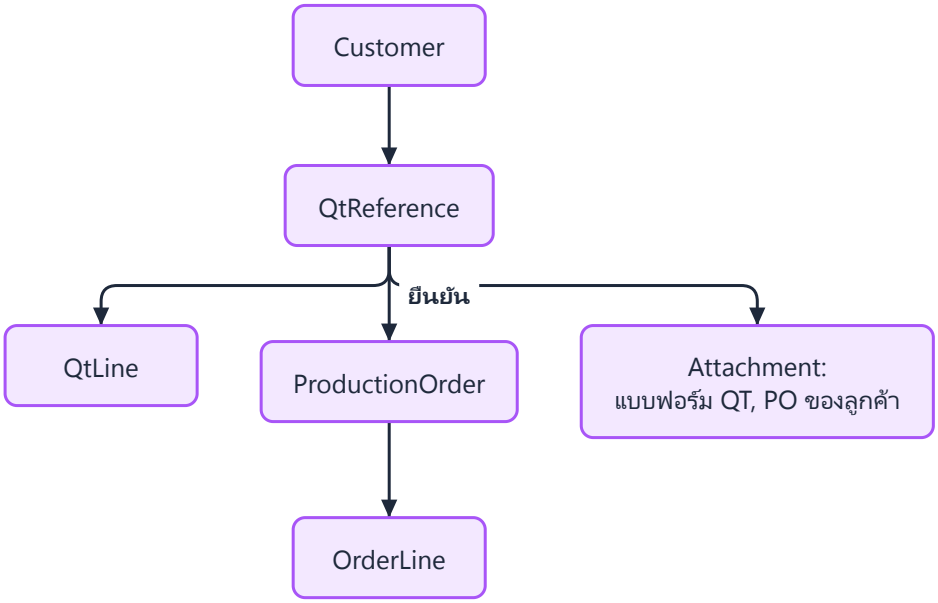
กฎของวงจรชีวิตที่เกี่ยวกับหลายตาราง

เหตุการณ์	ตารางที่ถูกเขียน	กฎ
สร้าง PO	PurchaseOrder, PurchaseOrderLine	สถานะ `Draft` เลือกได้เฉพาะวัตถุดิบของผู้จัดจำหน่ายที่เลือก (`SupplierMaterial`) ราคาต่อหน่วยเริ่มจาก `SupplierMaterial.unit_price` และผู้ใช้แก้ไขได้ `total` = ผลรวมของจำนวนเงินแต่ละรายการ
แก้ไข PO	PurchaseOrderLine	แก้ไขรายการได้เฉพาะตอน `Draft`
เปลี่ยนสถานะ PO	PurchaseOrder	`Draft` → `Sent` → `Received` เดินหน้าทีละขั้น ด้วยมือ
รับของตาม PO	PurchaseOrder.received_at, MaterialLot, StockChange	เพิ่มแต่ละรายการเข้าล็อตเป็นการเปลี่ยนแปลงสต็อกแบบ `Add` โดยเชื่อมด้วย `purchase_order_id` **ครั้งเดียวเท่านั้น**: การตั้งเป็น `Received` ซ้ำจะไม่เพิ่มอีก
จ่ายเงินตาม PO	PurchasePayment	ต้องมีสถานะ `Received` จ่ายเงินได้หนึ่งครั้งต่อหนึ่ง PO
อัปโหลดไฟล์	Attachment	ประเภท `PO_FILE`, `QUOTATION`, `EVIDENCE`, `OTHER` ไฟล์ PDF, PNG หรือ JPG ขนาดไม่เกิน 10 MB `PO_FILE` ใหม่จะแทนที่ไฟล์เดิม
ยืนยัน QT	QtReference.status, ProductionOrder, OrderLine, QtReference.production_order_id	ต้องมีลูกค้าและอย่างน้อยหนึ่งรายการ สร้างใบสั่งผลิตฉบับร่างหนึ่งใบจากรายการใน QT ทำได้ครั้งเดียว ไฟล์ PO ของลูกค้าจะถูกแนบกับใบสั่งผลิต และล็อกรายการใน QT
เริ่มผลิต	เหมือนเดิม	ไม่เปลี่ยน

ห่วงโซ่ของกระบวนการใหม่



รูปที่ 2. ห่วงโซ่ของกระบวนการใหม่: จัดซื้อ



รูปที่ 3. ห่วงโซ่ของกระบวนการใหม่: งานขาย